



FOR JAVA DEVELOPERS MOVING TO NESTJS

# Nest took 50 MB.

## Spring Boot to NestJS, episode 11: file uploads and downloads

The requirement transfers: bound the upload. The default does not.

Spring Boot refuses a 2 MB multipart file out of the box. NestJS on Express, with Multer, accepts 50 MB and keeps it in memory.

The heap barely moves while it happens, because a Node Buffer lives outside the V8 heap. Watch `arrayBuffers` and `rss`.

The fix is two decisions: a size bound, and a storage location. For downloads, stream rather than buffer.

### In scope

- NestJS on its default Express platform, uploads through Multer
- Which memory counter sees an upload, and why it reads twice the file
- `limits.fileSize` and `diskStorage`, as two separate decisions
- Buffered against streamed downloads, under concurrency

### Not in scope

- NestJS on Fastify: its file size cap defaults to 1 MiB (NestJS docs)
- Object storage uploads (S3 and friends)

Measurements are one machine, one local client. They show direction and scale, not constants.

## 01 The measurements

|                                |                                           |
|--------------------------------|-------------------------------------------|
| SPRING BOOT, 2 MB              | 413                                       |
| NESTJS + MULTER, 2 MB / 50 MB  | 201 / 201, inMemory true                  |
| ONE 50 MB UPLOAD: heapUsed     | 15.8 → 16.7 MB                            |
| ONE 50 MB UPLOAD: arrayBuffers | 0.1 → 100.1 MB                            |
| ONE 50 MB UPLOAD: rss          | 99.9 → 197.4 MB                           |
| WITH limits + diskStorage      | 512 KB 201 to disk, 50 MB 413             |
| 4 x 200 MB DOWNLOADS, PEAK     | buffered 800.3 MB, streamed 32.8 MB       |
| SAME, FORCED GC EVERY 20 MS    | buffered 800.1 MB, streamed 6.2 to 7.7 MB |

Same endpoint shape on both stacks, nothing configured unless stated.

## 02 Bound the upload, and choose where it goes

Two options, two decisions. `limits.fileSize` is the bound; without it Multer accepts any size. `diskStorage` decides where an accepted file goes; on its own it bounds nothing.

 `bounded.ts, the upload route (repository)`

```
import { FileInterceptor } from '@nestjs/platform-express';
import { diskStorage } from 'multer';

@Post('upload')
@UseInterceptors(FileInterceptor('file', {
  storage: diskStorage({ destination: uploadDir }),
  limits: { fileSize: 1024 * 1024 },
}))
upload(@UploadedFile() file: { size: number; buffer?: Buffer; path?: string }) {
  return {
    bytes: file?.size,
    inMemory: Buffer.isBuffer(file?.buffer),
    onDisk: typeof file?.path === 'string',
  };
}
```

 `the Spring Boot defaults it replaces`

```
spring.servlet.multipart.max-file-size=1MB
spring.servlet.multipart.max-request-size=10MB
spring.servlet.multipart.file-size-threshold=0B
```

`max-file-size` default 1MB; `max-request-size` 10MB; `file-size-threshold` 0B, "Threshold after which files are written to disk."

`spring-boot-servlet 4.1.1 configuration metadata`, read 2026-09-30.

`fileSize`: "For multipart forms, the max file size (in bytes)", default Infinity

`Multer 2.4.0 README and storage/memory.js`, read 2026-09-30.

### 03 Which counter sees an upload

Multer's memory storage pushes every chunk onto a list, then joins them with `Buffer.concat`, so for a moment the chunks and the joined copy both exist: consistent with 100 MB for a 50 MB file.

```
multer 2.4.0, storage/memory.js
```

```
file.stream.on('data', function (chunk) {  
  chunks.push(chunk)  
})  
file.stream.on('end', function () {  
  var buffer = Buffer.concat(chunks)
```

arrayBuffers "refers to memory allocated for ArrayBuffers and SharedArrayBuffers, including all Node.js Buffers."

Node.js 22 docs, `process.memoryUsage()`, read 2026-09-30.

heapUsed +0.9 MB, arrayBuffers +100.0 MB, rss +97.5 MB for one 50 MB upload

Run on a real machine, 2026-09-30, by `scripts/verify-uploads.sh` in the course repository.

### 04 Stream downloads rather than buffer them

A buffered response holds the whole file for as long as it is being sent, once per concurrent download. A streamed one passes chunks through. A plain memory reading shows about 30 MB for streaming; most of that is spent chunks the collector has not swept yet.

```
the two routes
```

```
@Get('buffered')  
async buffered() {  
  return new StreamableFile(await readFile(file));  
}  
  
@Get('streamed')  
streamed() {  
  return new StreamableFile(createReadStream(file));  
}
```

Four concurrent 200 MB downloads: buffered peak 800.3 MB, streamed 32.8 MB; with GC forced every 20 ms and the client at 40 MB/s, buffered 800.1 MB, streamed 6.2 to 7.7 MB over six runs

Run on a real machine, 2026-09-30, by `scripts/verify-uploads.sh` in the course repository.

## Before you ship an upload endpoint in NestJS

1. Set `limits.fileSize` on every `FileInterceptor`
2. Choose a storage: `diskStorage`, or memory only for small bounded files
3. Do not treat `diskStorage` as a limit
4. Watch `arrayBuffers` and `rss`, not only `heapUsed`
5. Stream downloads with `createReadStream` and `StreamableFile`
6. On Fastify, check its own multipart limits instead

### Everything else

|                                                                                                                                                                                                                            |                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Site</b><br><a href="https://code-with-sam-dev.github.io">https://code-with-sam-dev.github.io</a>                                     |  <b>Facebook</b><br><a href="https://www.facebook.com/codewithsamdev">https://www.facebook.com/codewithsamdev</a>     |
|  <b>YouTube</b><br><a href="https://www.youtube.com/@CodewithSam-Dev">https://www.youtube.com/@CodewithSam-Dev</a>                       |  <b>GitHub</b><br><a href="https://github.com/code-with-sam-dev">https://github.com/code-with-sam-dev</a>            |
|  <b>LinkedIn</b><br><a href="https://www.linkedin.com/company/code-with-sam-dev">https://www.linkedin.com/company/code-with-sam-dev</a> |  <b>Buy me a coffee</b><br><a href="https://buymeacoffee.com/codewithsam">https://buymeacoffee.com/codewithsam</a>  |
|  <b>x</b><br><a href="https://x.com/CodeWithSamDev">https://x.com/CodeWithSamDev</a>                                                    |  <b>Buy me sadza</b><br><a href="https://buymesadza.com/codewithsam">https://buymesadza.com/codewithsam</a>         |
|  <b>TikTok</b><br><a href="https://www.tiktok.com/@codewithsamdev">https://www.tiktok.com/@codewithsamdev</a>                           | <b>Every measurement, one command: <code>scripts/verify-uploads.sh</code></b><br><a href="https://github.com/code-with-sam-dev/spring-to-node">https://github.com/code-with-sam-dev/spring-to-node</a> |
|  <b>Instagram</b><br><a href="https://www.instagram.com/codewithsamdev">https://www.instagram.com/codewithsamdev</a>                    | <b>NestJS: file upload</b><br><a href="https://docs.nestjs.com/techniques/file-upload">https://docs.nestjs.com/techniques/file-upload</a>                                                              |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.